

RAPPORT D'EVALUATION TECHNIQUE

# QMS SaaS Pro

Evaluation de l'etat de realisation du systeme de gestion de la qualite conforme ISO 13485:2016, et analyse des perspectives de developpement pour les industries pharmaceutique et dispositifs medicaux.

Mai 2026

Z.ai - Division Qualite

Version 1.0

# Table des matieres

|                                                      |    |
|------------------------------------------------------|----|
| 1. Resume executif                                   | 3  |
| 2. Presentation du projet                            | 4  |
| 2.1 Stack technique . . . . .                        | 4  |
| 2.2 Architecture dual-mode . . . . .                 | 4  |
| 3. Etat de realisation par module                    | 5  |
| 3.1 Modules principaux (7) . . . . .                 | 5  |
| 3.2 Modules optionnels (8) . . . . .                 | 6  |
| 3.3 Composants d'infrastructure . . . . .            | 6  |
| 4. Qualite du code et tests                          | 7  |
| 4.1 Couverture de tests . . . . .                    | 7  |
| 4.2 Tests end-to-end . . . . .                       | 7  |
| 4.3 Points d'attention sur la qualite . . . . .      | 8  |
| 5. Evaluation de l'architecture                      | 9  |
| 5.1 Forces architecturales . . . . .                 | 9  |
| 5.2 Faiblesses et risques identifies . . . . .       | 9  |
| 6. Phases de developpement realisees                 | 10 |
| 7. Possibilites de developpement                     | 11 |
| 7.1 Priorite P0 - Imperatifs critiques . . . . .     | 11 |
| 7.2 Priorite P1 - Conformite reglementaire . . . . . | 11 |

|                                                          |    |
|----------------------------------------------------------|----|
| 7.3 Priorite P2 - Enrichissements fonctionnels . . . . . | 12 |
| 7.4 Priorite P3 - Optimisations techniques . . . . .     | 13 |
| 8. Recommandations                                       | 14 |
| 8.1 Actions immediates (semaine 1-2) . . . . .           | 14 |
| 8.2 Actions a court terme (mois 1-2) . . . . .           | 14 |
| 8.3 Actions a moyen terme (mois 3-6) . . . . .           | 14 |
| 9. Conclusion                                            | 15 |

# 1. Resume executif

Le projet QMS SaaS Pro est un systeme de gestion de la qualite conforme a la norme ISO 13485:2016, concu pour les industries pharmaceutique et des dispositifs medicaux. Developpe sur une architecture moderne Next.js 16 / React 19 / TypeScript, il integre 15 modules fonctionnels couvrant l'ensemble des exigences reglementaires, depuis la maitrise des documents jusqu'a la gestion des risques en passant par les CAPA, les non-conformites, les audits et les enregistrements de lots. Le projet a ete realise en 6 phases de developpement successives, aboutissant a un code base de plus de 38 000 lignes reparties dans 166 fichiers TypeScript/TSX.

L'etat actuel du projet est globalement solide : 824 tests unitaires et d'integration passent avec succes, le build de production compile sans erreur, et l'architecture dual-mode (Demo/Supabase) permet une demonstration fonctionnelle immediate tout en preservant la capacite de deploiement en production avec base de donnees PostgreSQL et Row Level Security. Neanmoins, plusieurs axes d'amelioration critiques ont ete identifies, notamment l'absence d'authentification fonctionnelle en mode demo, le manque de protection des routes par le middleware, et l'impossibilite actuelle de servir l'application via une URL accessible dans l'environnement sandbox.

| Indicateur                     | Valeur              | Statut       |
|--------------------------------|---------------------|--------------|
| Lignes de code source          | 38 237              | Termine      |
| Fichiers TypeScript/TSX        | 166                 | Termine      |
| Modules fonctionnels           | 15 / 15             | Termine      |
| Tests unitaires et integration | 824 / 824           | Termine      |
| Build production               | Compilation reussie | Termine      |
| Authentification               | Non configuree      | Non commence |
| Deploiement accessible         | Non fonctionnel     | Bloque       |

Tableau 1 : Indicateurs cles du projet QMS SaaS Pro

## 2. Presentation du projet

QMS SaaS Pro est une plateforme SaaS multi-tenant de gestion de la qualite, concue pour repondre aux exigences des normes ISO 13485:2016, ICH Q10, IVDR (Reglement UE 2017/746) et 21 CFR Part 11. La solution cible les entreprises des secteurs pharmaceutique, dispositifs medicaux, biotechnologie, diagnostics in vitro et produits combinants. L'architecture multi-locataire s'appuie sur Supabase avec Row Level Security (RLS), garantissant l'isolation complete des donnees entre organisations.

Le systeme offre un modele de controle d'accès basé sur les rôles (RBAC) comprenant 6 niveaux : Super Admin, Admin Qualite, Responsable Qualite, Utilisateur Qualite, Auditeur et Lecteur. Chaque rôle dispose de permissions granulaires permettant de limiter les actions sensibles aux seules personnes habilitées. La conformité 21 CFR Part 11 est assurée par un système de signatures électroniques avec génération de hash SHA-256 pour les opérations critiques telles que l'approbation de documents, la libération QA des lots, la completion de formations et la cloture des CAPA.

### 2.1 Stack technique

| Couche           | Technologie                                      | Version       |
|------------------|--------------------------------------------------|---------------|
| Framework        | Next.js (App Router)                             | 16.1          |
| UI               | React + TypeScript + TailwindCSS 4               | 19 / 5        |
| Composants       | shadcn/ui (style New York)                       | 42 composants |
| Base de donnees  | Prisma (SQLite dev) / Supabase (PostgreSQL prod) | 6.11 / 2.x    |
| State management | Zustand + TanStack Query + React Hook Form + Zod | 5 / 5 / 7 / 4 |
| Visualisation    | Recharts + TanStack Table                        | 2.x / 8.x     |
| Authentification | NextAuth + Supabase Auth (SSR)                   | 4.24          |
| Tests            | Vitest + Testing Library + Playwright + MSW      | 4.1 / latest  |
| i18n             | next-intl (EN/FR)                                | 4.3           |

Tableau 2 : Stack technique du projet

### 2.2 Architecture dual-mode

L'architecture du projet repose sur un mecanisme de detection automatique du mode de fonctionnement. Lorsque les variables d'environnement Supabase (NEXT\_PUBLIC\_SUPABASE\_URL et NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY) ne sont pas configurees ou contiennent des valeurs de placeholder, le systeme bascule automatiquement en mode Demo. Dans ce mode, les donnees sont stockees en memoire via un store Zustand avec des donnees de demonstration pre-chargees, permettant une evaluation fonctionnelle complete sans infrastructure externe.

En mode production (Supabase), le systeme active 15 services dedies qui implementent un CRUD complet avec scoping organisationnel automatique, conversion camelCase/snake\_case transparente, et enregistrement systematique dans le journal d'audit. Les politiques Row Level Security garantissent que chaque organisation ne peut acceder qu'a ses propres donnees, tandis que le rafraichissement de session est gere par le middleware Supabase pour maintenir la continuite d'authentification.

## 3. Etat de realisation par module

L'ensemble des 15 modules fonctionnels a ete implemente au cours des 6 phases de developpement. Chaque module dispose d'un composant React complet avec formulaires, tableaux de donnees, filtres, et integration des signatures electroniques. Les API routes correspondantes ont ete developpees pour chaque entite avec validation Zod et reponses standardisees. Les services Supabase offrent une couche d'accès aux données complete avec audit trail automatique.

### 3.1 Modules principaux (7)

| Module                 | Lignes | Fonctionnalites                                                | Avancement |
|------------------------|--------|----------------------------------------------------------------|------------|
| Controle des documents | 851    | CRUD complet, hierarchie, e-signatures, workflow d'approbation | Termine    |
| CAPA                   | 700    | Actions correctives/preventives, 5 Pourquoi, suivi efficacite  | Termine    |
| Non-conformites (NCR)  | 735    | NCR, investigation OOS, disposition, phase 1/2                 | Termine    |
| Audits                 | 700    | Internes/externes/fournisseurs, constatations, plans d'action  | Termine    |
| Formation              | 700    | Conformite formation, suivi retard, e-signature completion     | Termine    |

|            |      |                                               |                |
|------------|------|-----------------------------------------------|----------------|
| Rapports   | 1346 | 9 modeles de rapports, graphiques, export CSV | <b>Termine</b> |
| Conformite | 929  | ISO 13485, ICH Q10, IVDR, analyse des ecarts  | <b>Termine</b> |

Tableau 3 : Modules principaux - Etat de realisation

## 3.2 Modules optionnels (8)

| Module                   | Lignes | Fonctionnalites                                     | Avancement     |
|--------------------------|--------|-----------------------------------------------------|----------------|
| Gestion des risques      | 700    | FMEA, matrice 5x5, RPN, niveaux de risque           | <b>Termine</b> |
| Enregistrements de lots  | 882    | Etapas workflow, liberation QA, e-signature         | <b>Termine</b> |
| Fournisseurs             | 773    | Qualification, notation performance, certifications | <b>Termine</b> |
| Controle des changements | 832    | Workflow complet avec chemin de rejet               | <b>Termine</b> |
| Deviations               | 806    | Deviations planifiees/non planifiees                | <b>Termine</b> |
| OOS/OOT                  | 985    | Investigation phase 1/2 conforme FDA                | <b>Termine</b> |
| Formulaires dynamiques   | 1001   | Constructeur de modeles, remplissage instances      | <b>Termine</b> |
| Hierarchie documentaire  | 650    | Arbre visuel, filtrage par niveau                   | <b>Termine</b> |

Tableau 4 : Modules optionnels - Etat de realisation

## 3.3 Composants d'infrastructure

| Composant                  | Lignes | Description                                               | Avancement     |
|----------------------------|--------|-----------------------------------------------------------|----------------|
| Tableau de bord            | 800    | KPIs adaptes par industrie, poids de conformite           | <b>Termine</b> |
| Assistant de configuration | 972    | 6 etapes, selection industrie, configuration organisation | <b>Termine</b> |
| Navigation (Sidebar)       | 400    | Modules dynamiques, i18n, indicateur de mode              | <b>Termine</b> |

|                        |     |                                                   |         |
|------------------------|-----|---------------------------------------------------|---------|
| Gestion utilisateurs   | 738 | Roles/permissions, profils, invitations           | Termine |
| Recherche globale      | 200 | Recherche cross-module avec resultats contextuels | Termine |
| Signature electronique | 150 | Dialogue conforme 21 CFR Part 11, hash SHA-256    | Termine |

Tableau 5 : Composants d'infrastructure

## 4. Qualite du code et tests

### 4.1 Couverture de tests

La suite de tests du projet comprend 824 tests repartis dans 10 fichiers, couvrant les schemas de validation Zod, les check-lists de conformite, les services Supabase, le store de demonstration, le client API, les routes API, les utilitaires, les erreurs, les reponses et les composants partages. L'ensemble de ces tests passe avec succes en 5.55 secondes, ce qui temoigne d'une base de code stable et bien structuree. Le framework utilise est Vitest 4.1.5 avec environnement jsdom, accompagné de Testing Library et MSW pour le mocking des requetes reseau.

| Fichier de test               | Type        | Tests      | Lignes       |
|-------------------------------|-------------|------------|--------------|
| validation.test.ts            | Unite       | 228        | 1 699        |
| compliance-checklists.test.ts | Unite       | 171        | 1 587        |
| supabase-services.test.ts     | Unite       | 77         | 1 201        |
| demo-store.test.ts            | Unite       | 87         | 1 188        |
| api-client.test.ts            | Unite       | 73         | 734          |
| api-routes.test.ts            | Integration | 48         | 1 347        |
| shared-components.test.tsx    | Composant   | 52         | 400          |
| Autres (4 fichiers)           | Unite       | 88         | 806          |
| <b>TOTAL</b>                  |             | <b>824</b> | <b>9 362</b> |

Tableau 6 : Distribution des tests par fichier et type

### 4.2 Tests end-to-end



Un jeu de 7 tests E2E Playwright est configure pour valider les parcours critiques : acces a la page d'accueil, navigation par la barre laterale, affichage des KPIs du tableau de bord, fonctionnement des modules Documents, CAPA et NCR, et navigation cross-module. Ces tests fournissent une couche supplementaire de confiance dans le comportement global de l'application, bien que la couverture E2E puisse etre etendue pour couvrir davantage de scenarios metier critiques.

## 4.3 Points d'attention sur la qualite

Plusieurs aspects de la qualite du code meritent une attention particuliere. Premièrement, la configuration TypeScript est en mode permissif avec `noImplicitAny: false` et `ignoreBuildErrors: true`, ce qui signifie que des erreurs de type peuvent exister sans etre detectees lors du build. Bien que cela ait facilite le developpement rapide, il est recommande de renforcer progressivement la verification des types pour atteindre un niveau de securite adapté a un systeme GxP critique.

Deuxiemement, la couverture de tests actuelle se concentre principalement sur les couches inferieures (validation, services, store) et les routes API. Les composants React des 15 modules fonctionnels ne disposent pas encore de tests dedies, ce qui represente un risque significatif pour la maintenabilite et la fiabilite du front-end. Enfin, le fichier `mock-data.ts` de 1 171 lignes est un point de complexite elevee qui pourrait beneficier d'une refactoring en modules plus petits.

| Dimension              | Statut actuel        | Evaluation | Priorite |
|------------------------|----------------------|------------|----------|
| Tests unitaires        | 824 tests passants   | Termine    | -        |
| Tests E2E              | 7 tests smoke        | Partiel    | Haute    |
| Tests composants React | 1 fichier (52 tests) | Partiel    | Haute    |
| Typage TypeScript      | Mode permissif       | Partiel    | Moyenne  |
| Linting ESLint         | Configure            | Termine    | -        |
| Build production       | Reussi (9.8s)        | Termine    | -        |

Tableau 7 : Evaluation de la qualite du code

## 5. Evaluation de l'architecture

### 5.1 Forces architecturales

L'architecture du projet presente plusieurs forces notables qui constituent un socle solide pour le developpement futur. La separation nette entre mode Demo et mode Supabase permet une demonstration immediate sans dependance externe, tout en preservant la capacite de deploiement en production avec une base de donnees relationnelle et des politiques de securite au niveau des lignes. Le modele de service de base (BaseService) fournit une abstraction CRUD reutilisable avec scoping organisationnel automatique et journal d'audit integre, reduisant considerablement la duplication de code.

L'utilisation de Zod pour la validation des schemas a la fois cote client et cote serveur assure la coherence des contraintes metier. L'integration de 42 composants shadcn/ui garantit une interface utilisateur coherente et accessible, tandis que le systeme i18n (EN/FR) prefigure le support multilingue necessaire pour un deploiement international. L'architecture single-page application avec navigation par etat simplifie l'experience utilisateur tout en eliminant les problemes de synchronisation de route entre le client et le serveur.

### 5.2 Faiblesses et risques identifies

Plusieurs faiblesses architecturales ont ete identifiees et doivent etre adressees pour garantir la conformite reglementaire et la securite du systeme en production. La faiblesse la plus critique concerne l'authentification : le middleware actuel ne fait que rafraichir les sessions Supabase et ne redirige pas les utilisateurs non authentifies vers une page de connexion. Il n'existe aucune protection des routes basee sur les roles au niveau du middleware, ce qui signifie que toute personne ayant acces a l'URL peut potentiellement acceder a toutes les fonctionnalites.

Le second risque majeur concerne l'environnement de deploiement. L'application ne peut actuellement pas etre servie via une URL accessible dans l'environnement sandbox, ce qui bloque toute demonstration a distance. Le serveur Next.js, qu'il soit lance en mode developpement ou en mode production standalone, ne persiste pas dans l'environnement sandbox, et les tentatives de connexion locale via curl echouent en raison de l'isolation reseau.

Enfin, l'absence de page de connexion dediee, de flux d'inscription, de gestion de session visible et de deconnexion rend le systeme inutilisable en conditions reelles de production, meme si les fondations techniques (NextAuth, Supabase Auth) sont presentes dans les dependances. La validation reglementaire complete

(IQ/OQ/PQ) documentee dans le protocole de validation ne pourra etre executee que lorsque ces elements seront en place.

| Risque                                    | Impact   | Probabilite | Mitigation                                   |
|-------------------------------------------|----------|-------------|----------------------------------------------|
| Absence d'authentification fonctionnelle  | Critique | Certain     | Implementer NextAuth + pages login/logout    |
| Pas de protection des routes (middleware) | Critique | Certain     | Ajouter guards RBAC dans le middleware       |
| Deploiement inaccessible                  | Eleve    | Certain     | Configurer Vercel/Netlify ou Docker          |
| Typage permissif TypeScript               | Moyen    | Possible    | Activer strict mode progressivement          |
| Couverture de tests front-end limitee     | Moyen    | Possible    | Ajouter tests composants pour les 15 modules |
| Pas de tests de performance/charge        | Moyen    | Possible    | Implementer tests k6/Artillery               |

Tableau 8 : Matrice des risques identifies

## 6. Phases de developpement realisees

Le projet a ete developpe selon une approche incrementale en 6 phases, chacune apportant un ensemble coherent de fonctionnalites. Cette methode a permis de livrer progressivement des increments fonctionnels tout en maintenant la coherence architecturale globale. Le tableau ci-dessous resume les livrables de chaque phase ainsi que leur etat de completion.

| Phase   | Objectif             | Livrables cles                                                             | Statut         |
|---------|----------------------|----------------------------------------------------------------------------|----------------|
| Phase 1 | Services et guards   | BaseService, 15 services Supabase, demo-store, type system, validation Zod | <b>Termine</b> |
| Phase 2 | Hooks et API routes  | 6 hooks React, 14 entites CRUD API, validation, reponses standard          | <b>Termine</b> |
| Phase 3 | Composants front-end | 15 vues modules, AppLayout, Sidebar, SetupWizard, Dashboard                | <b>Termine</b> |
| Phase 4 | Integration Supabase | Client browser/server/admin, middleware, mode detection, migrations SQL    | <b>Termine</b> |

|         |                         |                                                                             |         |
|---------|-------------------------|-----------------------------------------------------------------------------|---------|
| Phase 5 | Conformite et tests     | 824 tests, check-lists ISO/ICH/IVDR, E2E Playwright, validation protocol    | Termine |
| Phase 6 | Polissage et industries | Configurations industrie, rapports avances, export CSV, OOS/OOT, deviations | Termine |

Tableau 9 : Phases de developpement et livrables

## 7. Possibilites de developpement

Malgre l'avancement considerable du projet, plusieurs axes de developpement prioritaires doivent etre adresses pour transformer le prototype actuel en produit deployable en production. Les sections suivantes detaille les possibilites de developpement classees par ordre de priorite, en distinguant les imperatifs reglementaires des ameliorations fonctionnelles et des optimisations techniques.

### 7.1 Priorite P0 - Imperatifs critiques

**Authentification et autorisation** : L'implementation complete du flux d'authentification est la priorite absolue. Cela comprend la creation d'une page de connexion/deconnexion, l'activation de NextAuth avec fournisseurs Supabase, la protection des routes par le middleware avec redirection vers la page de connexion pour les utilisateurs non authentifies, et l'ajout de guards base sur les roles au niveau du middleware pour empecher l'acces non autorise aux modules sensibles. Ce travail est indispensable pour toute mise en production, meme interne, car sans authentification, le systeme ne peut pas garantir la tracabilite requise par la norme ISO 13485 et le 21 CFR Part 11.

**Deploiement accessible** : La configuration d'un environnement de deploiement fonctionnel est essentielle pour permettre les tests d'acceptation, les demonstrations aux parties prenantes et la validation reglementaire. Les options recommandees incluent le deploiement sur Vercel (plateforme native pour Next.js), la containerisation Docker avec docker-compose pour un deploiement on-premise, ou l'utilisation d'un VPS avec un serveur Caddy deja configure dans le projet. Le fichier Caddyfile present dans le projet suggere que cette option a ete envisagee.

### 7.2 Priorite P1 - Conformite reglementaire

**Journal d'audit complet et immutable** : Bien que le systeme enregistre deja les operations CRUD dans le journal d'audit via le BaseService, il est necessaire de renforcer cette fonctionnalite pour atteindre la conformite 21 CFR Part 11. Cela

implique l'ajout d'horodatage avec resolution sub-seconde, la signature cryptographique des entrees d'audit pour empecher toute modification a posteriori, l'implementation d'un mecanisme de verification d'integrite du journal, et la creation d'une interface de consultation du journal d'audit avec filtres avancees et export.

**Workflows d'approbation complets** : Les workflows d'approbation des documents, CAPA et changements doivent etre enrichis pour supporter des scenarios multi-etapes avec notifications, escalades automatiques, et gestion des delegations d'approbation. Chaque transition d'etat doit declencher un enregistrement dans le journal d'audit avec la signature electronique de l'utilisateur ayant effectue l'action.

**Validation reglementaire IQ/OQ/PQ** : Le protocole de validation existe deja (PDF de 18 pages avec 113 cas de test), mais son execution effective necessite un environnement de deploiement stable et un systeme d'authentification fonctionnel. L'execution et la documentation des resultats de validation seront requises avant toute mise en service dans un environnement GxP.

## 7.3 Priorite P2 - Enrichissements fonctionnels

**Notifications et alertes** : L'implementation d'un systeme de notifications en temps reel permettrait d'informer les utilisateurs des evenements critiques : approbations en attente, CAPA depassees, formations arrives a expiration, audits planifies. L'integration de WebSocket (deja presente dans les dependances via le repertoire exemples/) permettrait la notification push, tandis qu'un systeme de notifications par email via un service tiers (SendGrid, AWS SES) completerait le dispositif pour les utilisateurs non connectes.

**Tableau de bord avance et analytics** : Le tableau de bord actuel affiche des KPIs statiques. L'ajout de graphiques d'evolution temporelle, de tendances de conformite, de comparaisons inter-périodes et d'indicateurs predictifs (risque de deviation, probabilite de non-conformite) apporterait une valeur considerable aux responsables qualite. L'integration de Recharts (deja dans les dependances) facilite cette evolution.

**API publique et integrations** : La creation d'une API REST documentee (OpenAPI/Swagger) permettrait l'integration avec des systemes tiers courants dans l'industrie pharmaceutique : ERP (SAP, Oracle), LIMS (Laboratory Information Management Systems), et systemes de gestion d'etiquettes. Cette ouverture transformerait le QMS d'un outil isole en un composant central de l'ecosysteme qualite.

## 7.4 Priorite P3 - Optimisations techniques

**Renforcement TypeScript** : L'activation progressive du mode strict TypeScript (noImplicitAny: true, ignoreBuildErrors: false) ameliorerait la securite du typage et la maintenabilite du code. Cette migration peut etre effectuee de maniere incrementale, fichier par fichier, en commençant par les modules critiques (validation, services, API routes).

**Performance et scalabilite** : L'optimisation des performances inclut l'implementation du rendu cote serveur (SSR) pour les pages a forte visibilite, la mise en cache des requetes frequentes via TanStack Query, la pagination cote serveur pour les grandes collections de documents, et l'optimisation des images et assets statiques. Des tests de charge avec k6 ou Artillery permettraient de valider la tenue du systeme sous contrainte.

**Accessibilite (WCAG 2.1 AA)** : L'audit d'accessibilite du front-end est necessaire pour garantir la conformite aux normes WCAG 2.1 AA, particulierement importante pour les applications utilisees dans des environnements reglementes. L'utilisation de shadcn/ui (base sur Radix UI) offre une bonne base d'accessibilite, mais des verifications specifiques sont requises pour les formulaires complexes et les tableaux de donnees.

| Priorite | Fonctionnalite                                 | Effort estime | Impact   |
|----------|------------------------------------------------|---------------|----------|
| P0       | Authentification NextAuth + pages login/logout | 3-5 jours     | Critique |
| P0       | Protection des routes (middleware RBAC)        | 2-3 jours     | Critique |
| P0       | Deploiement Vercel/Docker accessible           | 1-2 jours     | Critique |
| P1       | Journal d'audit immutable + signature crypto   | 5-7 jours     | Eleve    |
| P1       | Workflows d'approbation multi-etapes           | 7-10 jours    | Eleve    |
| P1       | Execution validation IQ/OQ/PQ                  | 3-5 jours     | Eleve    |
| P2       | Systeme de notifications (WebSocket + email)   | 7-10 jours    | Moyen    |
| P2       | Tableau de bord avance + analytics             | 5-7 jours     | Moyen    |
| P2       | API publique (OpenAPI/Swagger)                 | 5-7 jours     | Moyen    |
| P3       | TypeScript strict mode                         | 5-7 jours     | Moyen    |
| P3       | Tests de performance + SSR                     | 5-7 jours     | Faible   |

|    |                                 |           |        |
|----|---------------------------------|-----------|--------|
| P3 | Audit accessibilite WCAG 2.1 AA | 3-5 jours | Faible |
|----|---------------------------------|-----------|--------|

Tableau 10 : Feuille de route de developpement

## 8. Recommandations

### 8.1 Actions immediates (semaine 1-2)

La premiere action a entreprendre est la mise en place de l'authentification complete avec NextAuth et Supabase Auth. Cela implique la creation d'une page de connexion avec support des fournisseurs d'identite (email/mot de passe, Google, Microsoft), une page de gestion de profil, un mecanisme de deconnexion, et la protection de toutes les routes applicatives par le middleware. En parallele, le deploiement sur une plateforme accessible (Vercel recommande pour Next.js) doit etre configure pour permettre les tests et demonstrations a distance.

Il est egalement recommande de creer un fichier `.env.example` documentant l'ensemble des variables d'environnement requises, afin de faciliter la configuration par de nouveaux developpeurs ou administrateurs systeme. Ce fichier doit inclure les cles Supabase, les secrets NextAuth, l'URL de la base de donnees, et toute autre variable necessaire au fonctionnement du systeme.

### 8.2 Actions a court terme (mois 1-2)

Le renforcement du journal d'audit avec signature cryptographique et verification d'integrite est l'etape suivante prioritaire pour atteindre la conformite 21 CFR Part 11. L'implementation des workflows d'approbation multi-etapes avec notifications et escalades automatiques complementera cette conformite. L'execution du protocole de validation IQ/OQ/PQ dans un environnement stable permettra de documenter formellement la conformite du systeme.

L'extension de la couverture de tests aux composants React des 15 modules fonctionnels est egalement essentielle. Chaque vue de module devrait disposer au minimum de tests verifiant le rendu correct, les interactions utilisateur principales (creation, modification, suppression), et les cas limites (donnees manquantes, erreurs de validation). L'objectif devrait etre d'atteindre un minimum de 80% de couverture de code sur les composants critiques.

### 8.3 Actions a moyen terme (mois 3-6)

Le developpement des fonctionnalites avancees (notifications, tableau de bord analytics, API publique) constitue le troisieme palier de developpement. Ces

enrichissements transformeront le systeme d'un outil de gestion de la qualite basique en une plateforme collaborative et integrable, capable de s'insérer dans l'écosysteme numerique des entreprises pharmaceutiques et des dispositifs medicaux. L'ajout de langues supplementaires (allemand, espagnol, japonais) elargira le marche potentiel au-dela de la zone EN/FR actuellement supportee.

Enfin, la migration vers un mode strict TypeScript et la realisation d'un audit de performance et d'accessibilite permettront d'atteindre un niveau de qualite industrielle adapte au contexte reglementaire GxP. L'automatisation du pipeline CI/CD avec des controles qualite automatiques (lint, tests, build, securite) garantira la maintenabilite a long terme du projet.

| Dimension                | Note / 10 | Commentaire                                                                   |
|--------------------------|-----------|-------------------------------------------------------------------------------|
| Couverture fonctionnelle | 9.5       | 15/15 modules implements, tous operationnels en mode demo                     |
| Qualite du code          | 7.0       | Bon structure generale, mais typage permissif et quelques fichiers trop longs |
| Tests et validation      | 7.5       | 824 tests passants, mais couverture front-end insuffisante                    |
| Architecture             | 8.0       | Dual-mode elegant, services bien structures, mais lacunes securite            |
| Securite et conformite   | 4.0       | Fondations presentes mais auth absente, middleware incomplet                  |
| Deployabilite            | 3.5       | Build reussi mais pas d'environnement de deploiement fonctionnel              |
| Documentation            | 6.5       | Protocole de validation existe, mais manque de doc technique et API           |
| NOTE GLOBALE             | 6.6       | <b>Base solide avec lacunes critiques sur securite et deploiement</b>         |

Tableau 11 : Scoring global du projet

## 9. Conclusion

Le projet QMS SaaS Pro represente un effort de developpement considerable, avec 38 237 lignes de code couvrant 15 modules fonctionnels complets et 824 tests passants. La couverture fonctionnelle est excellente, l'architecture est bien pensee avec le mecanisme dual-mode, et les fondations techniques pour la conformite reglementaire sont en place. Cependant, le projet se situe



actuellement dans un état de prototype avancé plutôt que de produit déployable.

Les deux obstacles majeurs au déploiement en production sont l'absence d'authentification fonctionnelle et l'impossibilité de servir l'application via une URL accessible. Ces lacunes sont critiques dans le contexte réglementaire GxP où la traçabilité des accès et la sécurité des données sont des exigences incontournables. La résolution de ces problèmes (estimée à 6-10 jours de développement) transformerait le prototype en un système évaluable et potentiellement certifiable.

La feuille de route proposée s'articule en 4 niveaux de priorité (P0 à P3) couvrant une période de 6 mois. Les priorités P0 (authentification, déploiement) sont des préalables indispensables, les priorités P1 (conformité réglementaire) sont nécessaires pour toute utilisation dans un contexte GxP, et les priorités P2/P3 (enrichissements fonctionnels et optimisations) apportent la valeur ajoutée compétitive. Avec un investissement estimé de 2-3 mois supplémentaires, le projet peut atteindre un niveau de maturité suffisant pour une mise en production dans un environnement pharmaceutique ou dispositifs médicaux.